

期中测试

考试环境要求

1. 统一使用Jupyter Notebook完成编程任务（禁用VSCode等软件，避免插件造成不公平）
2. 考试用到的 Conda 虚拟环境会提前配置好 (CV_EXP)，在该虚拟环境下使用命令 `jupyter lab` 启动 Jupyter Kernel。
3. 进入网页 IDE 后，请在右上角选择对应的虚拟环境 Kernel `Python (CV_EXP)`
4. 在 Jupyter Lab 中，`Ctrl + 点击` 可跳转到函数或变量定义，可能用到的公式和函数说明均会在题干中给出。
5. 考试允许使用常用 Python 库 `numpy`、`matplotlib`、`sklearn` 等，但不允许直接调用现成的模型，例如 `sklearn.linear_model.LogisticRegression`、`sklearn.svm.SVC` 等。
6. 我们会提供之前的练习和作业题以及相应的答案作为参考资料，其他参考资料不允许携带（包括纸质资料、u盘等）。

考场纪律

1. 考试期间：
 - 禁止使用个人笔记本电脑
 - 手机需关机和随身物品统一放置于讲台前
2. 考试全程将进行录屏和录像监控，一经发现作弊行为将严肃处理
3. 考场安排：
 - 第 5-6 节课考生：完成答卷后可自习，但不得提前离开考场
 - 第 7-8 节课考生：提前十分钟开始考试，考试时间为 16:20-18:00
4. 建议所有同学都提前十分钟到达考场进行准备，检查设备是否有问题

注意事项

1. 严禁试题泄露：如发现第 5-6 节课考生泄题，该考生以及接受泄题考生的考试成绩均清零
2. 座位安排：考试前将公布详细座位表，请按指定 ID 就座
3. 考勤相关：到时会进行纸质签到，缺席的同学按 0 分处理，请假的同学也按缺席处理但可记考勤分，不设补考

请补全所有空白处代码，并运行后续测试与可视化单元。

第一题：SIFT 特征描述子计算

SIFT (Scale-Invariant Feature Transform, 尺度不变特征变换) 描述子是计算机视觉中最经典的局部特征之一，具有旋转、尺度和光照不变性。给定图像关键点邻域的梯度信息，SIFT 描述子通过统计局部梯度方向直方图来构建 128 维的特征向量。

描述子计算的主要步骤如下：

1. 在关键点周围取 16×16 的局部图像块 (patch)。
2. 计算 patch 内每个像素的梯度幅值 (magnitude) 与梯度方向 (orientation)。
3. 对梯度幅值施加以关键点为中心的高斯权重。
4. 将 16×16 的 patch 划分为 4×4 共 16 个子区域 (cell)，对每个 cell 统计 8 方向的梯度直方图。
5. 将 16 个 cell 的 8 维直方图拼接，得到 128 维描述子。
6. 对描述子进行 L2 归一化。

1. 问题描述

给定一个 18×18 的灰度图像块 `patch` (含 1 像素边界，用于计算中心 16×16 区域内每个像素的有限差分梯度)，你需要补全函数 `compute_sift_descriptor`，实现完整的 SIFT 特征描述子计算流程。

2. 公式

(1) 梯度计算 (有限差分)

对中心 16×16 区域中坐标为 (i, j) 的像素，使用其在 patch 中的相邻像素计算水平与垂直方向的梯度：

$$dx = \text{patch}[i + 1, j + 2] - \text{patch}[i + 1, j]$$

$$dy = \text{patch}[i + 2, j + 1] - \text{patch}[i, j + 1]$$

梯度幅值：

$$m(i, j) = \sqrt{dx^2 + dy^2}$$

梯度方向 (转换为角度，范围 $[0^\circ, 360^\circ)$)：

$$\theta(i, j) = \left(\arctan 2(dy, dx) \times \frac{180}{\pi} \right) \bmod 360$$

(2) 高斯权重

为了赋予距关键点越近的像素更高权重，对加权幅值施加以 16×16 区域中心为原点的二维高斯函数：

$$w(i, j) = \exp\left(-\frac{(i - c_y)^2 + (j - c_x)^2}{2\sigma^2}\right)$$

加权幅值：

$$\tilde{m}(i, j) = m(i, j) \times w(i, j)$$

(3) 方向直方图分箱

将 $[0^\circ, 360^\circ)$ 均匀划分为 $B = 8$ 个方向区间，每个区间宽度 $w = 45^\circ$ ：

$$\text{bin_idx} = \left\lfloor \frac{\theta(i, j)}{w} \right\rfloor \bmod B$$

将 $\tilde{m}(i, j)$ 累加到对应的方向 bin 中。

(4) 描述子构建与 L2 归一化

将 16 个 cell（每个 cell 8 维）拼接为 128 维向量后进行 L2 归一化：

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

3. 可能用到的函数

`np.sqrt(x)`：计算平方根 $\sqrt{x}$ 。

`np.arctan2(y, x)`：计算四象限反正切，返回弧度，范围 $(-\pi, \pi]$ ，注意参数顺序为（y分量，x分量）。

`np.pi`：圆周率 π 。

`np.exp(x)`：计算 e^x 。

`np.zeros((m, n))`：创建形状为 (m, n) 的全零数组。

`int(x)`：截断取整（对正数等价于向下取整）。

`a % b`：取模运算，如 `(-30) % 360 = 330`，`360 % 360 = 0`。

`array.flatten()`：将多维数组按行展平为一维数组。

`np.linalg.norm(v)`：计算向量的 L2 范数 $\|\mathbf{v}\|_2$ 。

4. 输入与输出

• 输入：

- `patch`：形状为 $(18, 18)$ 的二维 NumPy 数组，表示关键点邻域的灰度图像块（`dtype=float64`）。

• 输出：

- 一个长度为 128 的一维 NumPy 数组，表示经 L2 归一化后的 SIFT 描述子，满足 $\|\hat{\mathbf{v}}\|_2 = 1$ 。

5. Python 实现

你需要补全以下五处：

1. 补全1：梯度幅值 `mag[i, j]` 的计算
2. 补全2：梯度方向 `ori[i, j]` 的计算（结果为角度，范围 $[0^\circ, 360^\circ)$ ）
3. 补全3：高斯权重 `gauss_weight[i, j]` 的计算
4. 补全4：方向分箱索引 `bin_idx` 的计算
5. 补全5：对描述子进行 L2 归一化，得到 `descriptor_norm`

```
In [ ]: #####
#
#     test1    #
#
```

```
#####
import numpy as np

def compute_sift_descriptor(patch):
    """
    计算给定图像块的 SIFT 特征描述子。

    参数:
        patch (np.ndarray): 形状为 (18, 18) 的灰度图像块, dtype=float64。
            边界各 1 像素用于计算中心 16×16 区域的有限差分梯度。

    返回:
        np.ndarray: 长度为 128 的 L2 归一化 SIFT 描述子, dtype=float64。
    """
    H, W = 16, 16          # 有效计算区域大小
    n_bins = 8              # 方向直方图的 bin 数量
    bin_size = 360 / n_bins # 每个 bin 覆盖的角度范围 (45°)

    # ----- 步骤一: 计算梯度幅值与方向 -----
    mag = np.zeros((H, W)) # 梯度幅值矩阵
    ori = np.zeros((H, W)) # 梯度方向矩阵 (单位: 度)

    for i in range(H):
        for j in range(W):
            dx = patch[i+1, j+2] - patch[i+1, j]      # 水平梯度
            dy = patch[i+2, j+1] - patch[i, j+1]      # 垂直梯度
            #TODO(5')      # 补全1: 梯度幅值
            mag[i, j] = _____
            #TODO(8')      # 补全2: 梯度方向, 范围 [0°, 360°)
            ori[i, j] = _____

    # ----- 步骤二: 高斯加权 -----
    sigma = 8.0
    cy, cx = 7.5, 7.5      # 16×16 区域中心坐标
    gauss_weight = np.zeros((H, W))
    for i in range(H):
        for j in range(W):
            #TODO(7')      # 补全3: 根据高斯公式计算权重
            gauss_weight[i, j] = _____

    weighted_mag = mag * gauss_weight # 加权梯度幅值

    # ----- 步骤三: 统计方向直方图, 构建描述子 -----
    descriptor = np.zeros((4, 4, n_bins)) # 4×4 个 cell, 每个 cell 8 维直方图

    for bi in range(4):      # cell 行索引
        for bj in range(4):  # cell 列索引
            for pi in range(4): # cell 内像素行偏移
                for pj in range(4): # cell 内像素列偏移
                    i = bi * 4 + pi
                    j = bj * 4 + pj
                    angle = ori[i, j]
                    #TODO(5') # 补全4: 计算该像素所属的方向 bin 索引 (0~7)
                    bin_idx = _____

                    descriptor[bi, bj, bin_idx] += weighted_mag[i, j]

    # ----- 步骤四: 展平并 L2 归一化 -----
    desc = descriptor.flatten() # 展平为 128 维向量
    #TODO(5')      # 补全5: L2 归一化
```

```
descriptor_norm = _____  
  
return descriptor_norm
```

```
In [ ]: # ===== 测试 =====  
np.random.seed(2024)  
patch = np.random.randint(50, 200, size=(18, 18)).astype(np.float64)  
  
result = compute_sift_descriptor(patch)  
  
print("描述子维度:", result.shape)  
print("描述子 L2 范数:", np.linalg.norm(result))  
print("描述子前 8 维 (第 1 个 cell):")  
print(result[:8])
```

第二题：基于高斯金字塔的多尺度图像去噪与增强

1. 问题描述

在图像处理中，高斯滤波可以平滑图像，高斯金字塔可以将图像分解为不同尺度的结构信息。

本题要求你利用高斯金字塔构造一个简单的多尺度图像处理流程：

1. 对带噪图像构建 Gaussian Pyramid；
2. 根据相邻层之间的差异构建 detail pyramid；
3. 通过调整不同尺度 detail 的权重，实现：
 - 去噪：抑制高频噪声；
 - 增强：适当增强中低频细节；
4. 对比 noisy image、denoised image、enhanced image，并计算 MSE。

2. 可能用到的函数

- `np.asarray(a, dtype=np.float64)`：将输入数据转换为 NumPy 数组，并可通过 `dtype` 指定数据类型。
- `np.arange(start, stop, step=1)`：生成从 `start` 到 `stop` 之前的一维序列，`step` 控制步长。
- `np.meshgrid(x, y)`：根据两个一维坐标序列生成二维坐标网格，常用于构造坐标矩阵。
- `np.exp(x)`：对数组中每个元素计算指数函数。
- `np.sum(a, axis=None, keepdims=False)`：对数组元素求和。`axis` 指定求和维度，`keepdims=True` 可保留维度。
- `np.pad(array, pad_width, mode="reflect")`：对数组边界进行填充。`pad_width` 指定填充宽度，`mode="reflect"` 表示使用反射方式填充。
- `np.zeros(shape, dtype=np.float64)`：创建指定形状的全零数组，并可通过 `dtype` 指定数据类型。
- `np.repeat(a, repeats, axis=...)`：沿指定维度重复数组元素，可用于简单的尺寸放大。

- `np.clip(a, a_min, a_max)` : 将数组中的元素限制在 `[a_min, a_max]` 范围内。
- `plt.imshow(X, cmap="gray", vmin=0, vmax=1)` : 显示图像。
`cmap="gray"` 表示灰度显示, `vmin` 和 `vmax` 指定显示范围。
- `np.mean(a)` : 计算数组元素的平均值。

3. 公式

3.1 二维高斯核

大小为 $K \times K$ 、标准差为 σ 的高斯核为：

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

其中 x, y 是相对于卷积核中心的位置。

生成后需要进行归一化：

$$G \leftarrow \frac{G}{\sum_{x,y} G(x, y)}$$

保证卷积后整体亮度不会明显改变。

3.2 same 卷积

设输入图像为 I ，卷积核为 K ，输出图像为 O ，则：

$$O(i, j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} I_{\text{pad}}(i + u, j + v) K(u, v)$$

其中 I_{pad} 表示 padding 后的图像。

3.3 Gaussian Pyramid

第 0 层为原图：

$$G_0 = I$$

第 $l + 1$ 层由第 l 层先高斯滤波再下采样得到：

$$G_{l+1} = \text{Downsample}(G_l * K)$$

3.4 Detail Pyramid

本题中细节层定义为当前尺度图像与下一尺度图像上采样后的差：

$$D_l = G_l - \text{Upsample}(G_{l+1})$$

其中 D_l 保留了第 l 层中无法由更粗尺度表示的细节信息。

3.5 加权重建

从最粗尺度 G_L 开始逐层上采样，并加回细节层：

$$\hat{G}_l = \text{Upsample}(\hat{G}_{l+1}) + w_l D_l$$

当所有 $w_l = 1$ 时，应当能基本重建原始输入图像。

当较细尺度的 w_l 较小时，可以抑制高频噪声；当中低频尺度的 w_l 略大于 1 时，可以增强结构细节。

3.6 均方误差 MSE

$$\text{MSE}(I, \hat{I}) = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W \left(I(i, j) - \hat{I}(i, j) \right)^2$$

4. 输入输出

输入

- 一张灰度图像 `clean_img`，范围为 `[0, 1]`；
- 一张加噪图像 `noisy_img`；
- 金字塔层数 `levels`；
- 高斯核大小 `kernel_size`；
- 高斯核标准差 `sigma`；
- 不同尺度的细节权重 `detail_weights`。

输出

- Gaussian Pyramid；
- Detail Pyramid；
- 去噪图像 `denoised_img`；
- 增强图像 `enhanced_img`；
- noisy / denoised / enhanced 的 MSE 对比；
- 可视化结果。

5. 需要补全的 Python 函数

1. `gaussian_kernel(size, sigma)`：生成归一化二维高斯核；
2. `conv2d_same(img, kernel)`：实现 same 卷积；
3. `downsample(img)`：实现 2 倍下采样；
4. `upsample(img, target_shape)`：上采样到指定尺寸；
5. `build_gaussian_pyramid(img, levels, kernel)`：构建高斯金字塔；
6. `build_detail_pyramid(g_pyr)`：构建细节金字塔；
7. `reconstruct_from_detail_pyramid(base, detail_pyr, detail_weights)`：根据细节权重逐层重建图像。

注意：

- 不要直接调用现成的金字塔函数；
- 卷积输出大小应与输入图像一致；
- `detail_pyr` 中的每一层应与对应的 `G_l` 尺寸一致；

- `detail_weights` 的长度应与 `detail_pyr` 的长度一致;
- 最终输出图像建议使用 `np.clip(img, 0, 1)` 限制到 `[0, 1]`。

```
In [ ]: #####
#                                     #
#      test2                         #
#                                     #
#####
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

def create_test_image(size=128):
    """
    生成一张用于测试的灰度图像。
    图像包含平滑背景、矩形、圆形和条纹结构，便于观察去噪和增强效果。
    """
    y, x = np.mgrid[0:size, 0:size]
    img = 0.25 + 0.35 * (x / size) + 0.20 * (y / size)

    # 添加矩形区域
    img[25:75, 18:62] += 0.25

    # 添加圆形区域
    circle = (x - 88) ** 2 + (y - 70) ** 2 < 24 ** 2
    img[circle] += 0.28

    # 添加中频条纹
    stripe_region = (y > 82) & (y < 112) & (x > 18) & (x < 110)
    img[stripe_region] += 0.08 * np.sin(x[stripe_region] * 0.7)

    return np.clip(img, 0, 1)

clean_img = create_test_image(size=128)
noise = np.random.normal(loc=0.0, scale=0.08, size=clean_img.shape)
noisy_img = np.clip(clean_img + noise, 0, 1)

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(clean_img, cmap="gray", vmin=0, vmax=1)
plt.title("Clean image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(noisy_img, cmap="gray", vmin=0, vmax=1)
plt.title("Noisy image")
plt.axis("off")
plt.show()
```

```
In [ ]: def gaussian_kernel(size, sigma):
    """
    生成二维高斯卷积核。

    参数:
        size: int, 卷积核大小, 要求为奇数, 例如 3、5、7。
        sigma: float, 高斯分布标准差。

    返回:
```



```

        kernel: shape 为 (size, size) 的二维数组, 且 kernel.sum() 应接近 1。
    """
    if size % 2 == 0:
        raise ValueError("size must be odd.")

    radius = size // 2
    ax = np.arange(-radius, radius + 1)
    xx, yy = np.meshgrid(ax, ax)

    # TODO(5') 1: 根据二维高斯公式生成 kernel, 并进行归一化。
    kernel = _____
    normed_kernel = _____
    return normed_kernel

def conv2d_same(img, kernel):
    """
    对灰度图像进行 same 卷积, 输出大小与输入相同。

    参数:
        img: shape 为 (H, W) 的二维数组。
        kernel: shape 为 (K, K) 的二维卷积核。

    返回:
        out: shape 为 (H, W) 的卷积结果。
    """
    img = np.asarray(img, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)

    H, W = img.shape
    kH, kW = kernel.shape

    # TODO(3') 2: 计算 padding, 并使用 reflect 方式填充图像。
    pad_h = _____
    pad_w = _____
    padded = _____

    out = np.zeros((H, W), dtype=np.float64)

    # TODO(5') 3: 逐像素提取局部窗口, 完成卷积求和。
    for i in range(H):
        for j in range(W):
            window = _____
            out[i, j] = _____

    return out

def downsample(img):
    """
    对图像进行 2 倍下采样。
    这里假设输入图像在调用该函数前已经完成低通滤波。

    参数:
        img: shape 为 (H, W) 的二维数组。

    返回:
        下采样后的图像。
    """
    # TODO(3') 4: 实现简单的下采样, 直接取偶数行和偶数列。

```

```

return _____

def upsample(img, target_shape):
    """
    使用简单的最近邻方式将图像上采样到 target_shape。

    参数:
        img: shape 为 (h, w) 的二维数组。
        target_shape: tuple, 目标尺寸 (H, W)。

    返回:
        up: shape 为 target_shape 的二维数组。
    """
    H, W = target_shape
    # TODO(4') 5: 实现简单的上采样, 使用 np.repeat 将每个像素复制成 2x2 块。
    up = _____
    return _____

def build_gaussian_pyramid(img, levels, kernel):
    """
    构建 Gaussian Pyramid。

    参数:
        img: 输入图像。
        levels: 金字塔层数, 包括原图层。
        kernel: 高斯卷积核。

    返回:
        g_pyr: list, g_pyr[0] 是原图, 后续层逐渐变小。
    """
    g_pyr = [np.asarray(img, dtype=np.float64)]
    current = g_pyr[0]

    # TODO(4') 6: 每一层先高斯滤波, 再下采样。
    for _ in range(1, levels):
        blurred = _____
        current = _____
        g_pyr.append(current)

    return g_pyr

def build_detail_pyramid(g_pyr):
    """
    根据 Gaussian Pyramid 构建 Detail Pyramid。

    参数:
        g_pyr: Gaussian Pyramid。

    返回:
        detail_pyr: list, 其中 detail_pyr[l] = G_l - upsample(G_{l+1})。
    """
    detail_pyr = []

    # TODO(3') 7: 计算相邻 Gaussian 层之间的 detail residual。
    for l in range(len(g_pyr) - 1):
        up = _____
        detail = _____

```

```

        detail_pyr.append(detail)

    return detail_pyr

def reconstruct_from_detail_pyramid(base, detail_pyr, detail_weights):
    """
    从最粗层 base 和 detail pyramid 重建图像。

    参数:
        base: Gaussian Pyramid 的最后一层, 即最粗尺度图像。
        detail_pyr: detail pyramid, 顺序为从细到粗。
        detail_weights: 每个 detail 层的权重, 长度与 detail_pyr 相同。

    返回:
        reconstructed: 重建图像。
    """
    if len(detail_pyr) != len(detail_weights):
        raise ValueError("detail_weights must have the same length as det

    current = np.asarray(base, dtype=np.float64)

    # TODO(3') 8: 从最粗的 detail 开始逐层上采样, 并加权加回 detail。
    for detail, weight in zip(detail_pyr[::-1], detail_weights[::-1]):
        current = _____
        current = _____

    return current

```

```

In [ ]: kernel = gaussian_kernel(size=5, sigma=1.2)

print("Gaussian kernel:")
print(kernel)
print("kernel sum:", kernel.sum())

levels = 4
g_pyr = build_gaussian_pyramid(noisy_img, levels=levels, kernel=kernel)
detail_pyr = build_detail_pyramid(g_pyr)

print("\nGaussian Pyramid shapes:")
for i, g in enumerate(g_pyr):
    print(f"G_{i}: {g.shape}")

print("\nDetail Pyramid shapes:")
for i, d in enumerate(detail_pyr):
    print(f"D_{i}: {d.shape}")

# 验证: 当所有 detail weight 为 1 时, 应当可以重建 noisy_img
reconstructed_noisy = reconstruct_from_detail_pyramid(
    base=g_pyr[-1],
    detail_pyr=detail_pyr,
    detail_weights=[1.0] * len(detail_pyr)
)

reconstruction_error = np.mean((reconstructed_noisy - noisy_img) ** 2)
print("\nReconstruction MSE with all weights = 1:", reconstruction_error)

```

```

In [ ]: # 设计两组 detail 权重
        # detail_pyr[0] 对应最细尺度, 通常包含较多高频噪声。

```

```
# 去噪：降低最细尺度权重，适当保留中低频结构。
denoise_weights = [0.15, 0.55, 0.90]

# 增强：适当增强中尺度与粗尺度细节，但不要过度放大最高频噪声。
enhance_weights = [0.80, 1.25, 1.15]

denoised_img = reconstruct_from_detail_pyramid(
    base=g_pyr[-1],
    detail_pyr=detail_pyr,
    detail_weights=denoise_weights
)

enhanced_img = reconstruct_from_detail_pyramid(
    base=g_pyr[-1],
    detail_pyr=detail_pyr,
    detail_weights=enhance_weights
)

denoised_img = np.clip(denoised_img, 0, 1)
enhanced_img = np.clip(enhanced_img, 0, 1)

mse_noisy = np.mean((clean_img - noisy_img) ** 2)
mse_denoised = np.mean((clean_img - denoised_img) ** 2)
mse_enhanced = np.mean((clean_img - enhanced_img) ** 2)

print(f"MSE(noisy)      : {mse_noisy:.6f}")
print(f"MSE(denoised)   : {mse_denoised:.6f}")
print(f"MSE(enhanced)    : {mse_enhanced:.6f}")

plt.figure(figsize=(14, 4))
plt.subplot(1, 4, 1)
plt.imshow(clean_img, cmap="gray", vmin=0, vmax=1)
plt.title("Clean")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(noisy_img, cmap="gray", vmin=0, vmax=1)
plt.title("Noisy")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(denoised_img, cmap="gray", vmin=0, vmax=1)
plt.title("Denoised")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(enhanced_img, cmap="gray", vmin=0, vmax=1)
plt.title("Enhanced")
plt.axis("off")
plt.show()

plt.figure(figsize=(12, 4))
for i, detail in enumerate(detail_pyr):
    plt.subplot(1, len(detail_pyr), i + 1)
    plt.imshow(detail, cmap="gray")
    plt.title(f"Detail D_{i}")
    plt.axis("off")
plt.show()
```

第三题：手写数字分类

1. 问题描述

本题使用 `sklearn.datasets.load_digits()` 中的 8×8 手写数字数据集。你需要从零实现一个通用线性分类器框架，并在同一训练框架下分别实现：

1. **Softmax Classifier**;
2. **Multiclass SVM Classifier**。

Softmax 和 SVM 的模型形式相同，都是：

$$S = XW + b$$

它们的区别主要在于 **loss function** 和对应梯度。

因此，本题要求你实现一个 `LinearClassifier` 基类，并让 `SoftmaxClassifier` 与 `SVMClassifier` 继承它。

2. 可能用到的函数

- `load_digits()`：从 `sklearn.datasets` 中加载内置的手写数字数据集，返回图像数据、标签等信息。
 - `train_test_split(X, y, test_size=..., random_state=..., stratify=...)`：将数据集划分为训练集和测试集。
 - `reshape(...)`：改变数组形状，例如将图像展开为一维向量，或将图像划分为若干小块。
 - `np.sum(a, axis=..., keepdims=...)`：沿指定维度求和。
`keepdims=True` 可保留原维度，便于后续广播计算。
 - `np.mean(a, axis=..., keepdims=...)`：沿指定维度计算平均值。
 - `np.std(a, axis=..., keepdims=...)`：沿指定维度计算标准差。
 - `np.diff(a, axis=...)`：沿指定维度计算相邻元素之间的差值。
 - `np.abs(a)`：计算数组中每个元素的绝对值。
 - `np.arange(N)`：生成从 0 到 N-1 的整数序列。
 - `np.concatenate(arrays, axis=...)`：沿指定维度拼接多个数组。
 - `np.max(a, axis=..., keepdims=...)`：沿指定维度取最大值。
 - `np.exp(a)`：对数组中每个元素计算指数函数。
 - `np.log(a)`：对数组中每个元素计算自然对数。
 - `np.maximum(x, y)`：逐元素比较并返回较大值。
 - `np.random.default_rng(seed)`：创建一个随机数生成器，`seed` 用于固定随机结果。
 - `rng.permutation(N)`：生成 0 到 N-1 的随机排列。
 - `np.argmax(scores, axis=1)`：沿指定维度返回最大值所在的索引，常用于得到预测类别。
 - `np.linalg.norm(W)`：计算矩阵或向量的范数。
-

3. 特征设计说明

本题需要比较两种输入特征：**原始像素特征**和**扩展特征**。设一张数字图像为 $I \in \mathbb{R}^{8 \times 8}$ 。

原始像素特征直接将图像展开：

$$x_{raw} = \text{flatten}(I) \in \mathbb{R}^{64}$$

扩展特征是在原始像素的基础上，额外加入几类简单的形状统计信息，使线性分类器更容易利用数字的笔画分布。

3.1. 二值笔画特征：

根据图像平均灰度得到二值图 B ，再展开为 64 维向量。

$$B_{i,j} = \begin{cases} 1, & I_{i,j} > \text{mean}(I) \\ 0, & I_{i,j} \leq \text{mean}(I) \end{cases}$$

3.2. 分区强度特征：

将 8×8 图像划分为 4×4 个 2×2 小块，每个小块取平均值，得到 16 维特征。

$$P_{u,v} = \frac{1}{4} \sum_{i=2u}^{2u+1} \sum_{j=2v}^{2v+1} I_{i,j}, \quad u, v \in 0, 1, 2, 3$$

3.3. 边缘投影特征：

计算相邻像素的绝对差分，并分别沿行、列方向求平均，得到 14 维特征。

$$D_x(i, j) = |I_{i,j+1} - I_{i,j}|, \quad i = 1, \dots, 8, j = 1, \dots, 7$$

$$D_y(i, j) = |I_{i+1,j} - I_{i,j}|, \quad i = 1, \dots, 7, j = 1, \dots, 8$$

$$p_x(j) = \frac{1}{8} \sum_{i=1}^8 D_x(i, j), \quad j = 1, \dots, 7$$

$$p_y(i) = \frac{1}{8} \sum_{j=1}^8 D_y(i, j), \quad i = 1, \dots, 7$$

3.4. 强度重心特征：

根据像素强度计算数字在行方向和列方向的重心位置，即灰度分布中心，得到 2 维特征。

$$c_x = \frac{\sum_{i,j} j I_{i,j}}{\sum_{i,j} I_{i,j} + \epsilon}, \quad c_y = \frac{\sum_{i,j} i I_{i,j}}{\sum_{i,j} I_{i,j} + \epsilon}$$

最终扩展特征为：

$$\mathbf{x} = [x_{raw}, x_{binary}, x_{block}, x_{edge}, x_{center}]$$

各部分维度分别为 64, 64, 16, 14, 2，因此扩展特征总维度为：

$$64 + 64 + 16 + 14 + 2 = 160$$

4. 分类器公式

4.1 线性得分

给定输入矩阵 $X \in \mathbb{R}^{N \times D}$ ，权重矩阵 $W \in \mathbb{R}^{D \times C}$ ，偏置 $b \in \mathbb{R}^C$ ：

$$S = XW + b$$

其中 $C = 10$ 。

4.2 Softmax loss

$$P_{ij} = \frac{\exp(S_{ij})}{\sum_{k=1}^C \exp(S_{ik})}$$

计算时需要先减去每一行最大值：

$$S'_{ij} = S_{ij} - \max_k S_{ik}$$

交叉熵损失为：

$$\mathcal{L}_{softmax} = -\frac{1}{N} \sum_{i=1}^N \log P_{i, y_i} + \frac{1}{2} \lambda \|W\|_2^2$$

Softmax + Cross Entropy 对得分的梯度：

$$\frac{\partial \mathcal{L}}{\partial S} = \frac{1}{N} (P - Y)$$

由于：

$$S = XW + b$$

所以：

$$\frac{\partial \mathcal{L}}{\partial W} = X^T \frac{\partial \mathcal{L}}{\partial S} + \lambda W$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial S_i}$$

其中， $Y \in \mathbb{R}^{N \times C}$ 是标签对应的 one-hot 矩阵。

4.3 Multiclass SVM loss

对第 i 个样本，设正确类别得分为 S_{i, y_i} ，margin 为：

$$m_{ij} = \max(0, S_{ij} - S_{i, y_i} + \Delta), \quad j \neq y_i$$

其中一般取 $\Delta = 1$ 。

整体损失为：

$$\mathcal{L}_{svm} = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} m_{ij} + \frac{1}{2} \lambda \|W\|_2^2$$

SVM 梯度可先构造一个 mask 矩阵：

$$M_{ij} = 1 \quad \text{if } m_{ij} > 0, j \neq y_i$$

并令：

$$M_{i,y_i} = - \sum_{j \neq y_i} M_{ij}$$

则：

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{N} X^T M + \lambda W, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_i M_i$$

5. 输入输出

输入

- `images` : shape 为 `(N, 8, 8)` 的数字图像；
- `y` : shape 为 `(N,)` 的整数标签，范围为 0 到 9；
- 分类器类型： `SoftmaxClassifier` 或 `SVMClassifier`；
- 学习率 `lr`；
- 正则强度 `reg`；
- 训练轮数 `epochs`；
- batch 大小 `batch_size`。

输出

- 训练好的权重 `W` 和偏置 `b`；
- loss 曲线；
- train accuracy；
- test accuracy；
- Softmax 与 SVM 的对比结果；
- 不同正则强度下的结果表格。

6. 需要补全的 Python 函数

你需要补全关键代码：

1. `build_features(images, use_extra_features=True)` : 构造扩展结构特征；
2. `softmax(scores)` : 计算数值稳定版 Softmax；
3. `cross_entropy_loss(probs, y)` : 计算交叉熵损失；
4. `softmax_loss_and_gradient(X, scores, y, W, reg)` : 计算 Softmax loss 及梯度；

5. `svm_loss_and_gradient(X, scores, y, W, reg, delta=1.0)` : 计算 SVM hinge loss 及梯度;
6. `LinearClassifier.fit(X, y)` : 实现 mini-batch SGD 中的 loss 调用与参数更新。

注意:

- `LinearClassifier` 负责共同训练流程, 包括参数初始化、mini-batch SGD、预测和准确率计算;
- `SoftmaxClassifier` 和 `SVMClassifier` 复用同一训练流程, 只是 loss 和 gradient 不同;
- 标准化只能使用训练集统计量;
- 不允许调用 `sklearn` 中的分类器直接训练。

```
In [ ]: #####
#                                     #
#      test3                         #
#                                     #
#####
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
images = digits.images.astype(np.float64)
labels = digits.target.astype(np.int64)

print("images shape:", images.shape)
print("labels shape:", labels.shape)

plt.figure(figsize=(8, 2))
for i in range(8):
    plt.subplot(1, 8, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(str(labels[i]))
    plt.axis("off")
plt.show()
```

```
In [ ]: def build_features(images, use_extra_features=True):
    """
    构造数字图像特征。

    参数:
        images: shape 为 (N, 8, 8) 的图像数组。
        use_extra_features:
            False: 只返回原始像素特征, shape 为 (N, 64)。
            True : 返回原始像素 + 二值笔画 + 4×4 分区强度 + 边缘投影 + 重心特征,

    返回:
        X: shape 为 (N, D) 的特征矩阵。
    """
    images = np.asarray(images, dtype=np.float64)
    N = images.shape[0]

    raw = images.reshape(N, -1)

    if not use_extra_features:
```

```

        return raw

    binary = (images > 8.0).astype(np.float64).reshape(N, -1)

    # TODO(4') 1: 计算 4×4 分区强度, 每个分区大小为 2×2。
    block_sum = _____

    # TODO(4') 2: 计算简单边缘投影。提示: 先用 np.diff, 再对绝对值求平均。
    gx = _____ # (N, 8, 7)
    gy = _____ # (N, 7, 8)
    edge_x = _____ # (N, 7)
    edge_y = _____ # (N, 7)

    # TODO(2') 3: 计算强度重心位置。
    ink = images.sum(axis=(1, 2), keepdims=True) + 1e-8
    row_coords = np.arange(8).reshape(1, 8, 1)
    col_coords = np.arange(8).reshape(1, 1, 8)
    center_y = _____
    center_x = _____
    center = np.concatenate([center_y, center_x], axis=1)

    X = np.concatenate([raw, binary, block_sum, edge_x, edge_y, center],
                        return X

def standardize_train_test(X_train, X_test, eps=1e-8):
    """
    使用训练集的均值和标准差对训练集、测试集做标准化。
    """
    mean = X_train.mean(axis=0, keepdims=True)
    std = X_train.std(axis=0, keepdims=True)

    X_train_std = (X_train - mean) / (std + eps)
    X_test_std = (X_test - mean) / (std + eps)

    return X_train_std, X_test_std

def softmax(scores):
    """
    计算数值稳定版 softmax。

    参数:
        scores: shape 为 (N, C) 的得分矩阵。

    返回:
        probs: shape 为 (N, C) 的概率矩阵, 每行和为 1。
    """
    # TODO(5') 4: 每行减去最大值后计算 softmax。
    shifted = _____
    exp_scores = _____
    probs = _____
    return probs

def cross_entropy_loss(probs, y):
    """
    计算交叉熵损失, 不包含正则项。
    """
    N = y.shape[0]

```

```

# TODO(4') 5: 取出真实类别概率并计算平均交叉熵。
correct_probs = _____
loss = _____
return loss

def softmax_loss_and_gradient(X, scores, y, W, reg):
    """
    计算 Softmax loss 以及 W、b 的梯度。
    """
    probs = softmax(scores)
    data_loss = cross_entropy_loss(probs, y)
    loss = data_loss + 0.5 * reg * np.sum(W * W)

    N = X.shape[0]

    # TODO(8') 6: 计算 Softmax 对 scores 的梯度, 并进一步计算 dW、db, 并添加正则
    dscores = probs.copy()
    dscores[np.arange(N), y] = _____
    dscores = _____

    dW = _____
    db = _____

    return loss, dW, db

def svm_loss_and_gradient(X, scores, y, W, reg, delta=1.0):
    """
    计算 multiclass SVM hinge loss 以及 W、b 的梯度。
    """
    N = X.shape[0]

    # TODO(5') 7: 计算 margin 和 SVM loss。
    correct_scores = _____
    margins = _____
    margins[np.arange(N), y] = 0.0

    data_loss = _____
    loss = data_loss + 0.5 * reg * np.sum(W * W)

    # TODO(4') 8: 根据 margin 构造梯度 mask, 并计算 dW、db, 并添加正则化项。
    binary = _____
    row_sum = np.sum(binary, axis=1)
    binary[np.arange(N), y] = _____

    dW = _____
    db = _____

    return loss, dW, db

class LinearClassifier:
    """
    通用线性分类器基类。

    子类只需要实现 loss_and_gradient()。
    """
    def __init__(self, lr=0.1, reg=1e-4, epochs=80, batch_size=64, random

```

```

self.lr = lr
self.reg = reg
self.epochs = epochs
self.batch_size = batch_size
self.random_state = random_state
self.W = None
self.b = None
self.loss_history = []

def _init_params(self, D, C):
    rng = np.random.default_rng(self.random_state)
    self.W = 0.01 * rng.standard_normal((D, C))
    self.b = np.zeros(C, dtype=np.float64)

def loss_and_gradient(self, X_batch, y_batch):
    raise NotImplementedError

def fit(self, X, y):
    """
    使用 Mini-batch SGD 训练线性分类器。
    """
    rng = np.random.default_rng(self.random_state)
    N, D = X.shape
    C = int(np.max(y)) + 1

    self._init_params(D, C)
    self.loss_history = []

    for epoch in range(self.epochs):
        indices = rng.permutation(N)
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        last_loss = None

        for start in range(0, N, self.batch_size):
            end = min(start + self.batch_size, N)
            X_batch = X_shuffled[start:end]
            y_batch = y_shuffled[start:end]

            # TODO(4') 9: 调用当前分类器的 loss_and_gradient, 并用 SGD 更
            loss, dW, db = _____
            self.W = _____
            self.b = _____

            last_loss = loss

        self.loss_history.append(last_loss)

    return self

def predict(self, X):
    """
    预测类别标签。
    """
    scores = X @ self.W + self.b
    return np.argmax(scores, axis=1)

def accuracy(self, X, y):
    """

```

```

        计算分类准确率。
        """
        pred = self.predict(X)
        return np.mean(pred == y)

class SoftmaxClassifier(LinearClassifier):
    def loss_and_gradient(self, X_batch, y_batch):
        scores = X_batch @ self.W + self.b
        return softmax_loss_and_gradient(X_batch, scores, y_batch, self.W

class SVMClassifier(LinearClassifier):
    def loss_and_gradient(self, X_batch, y_batch):
        scores = X_batch @ self.W + self.b
        return svm_loss_and_gradient(X_batch, scores, y_batch, self.W, se

```

```

In [ ]: # 简单单元测试: 检查形状、softmax 性质以及 SVM loss/gradient 形状
X_raw = build_features(images, use_extra_features=False)
X_extra = build_features(images, use_extra_features=True)

print("Raw feature shape:", X_raw.shape)
print("Extra feature shape:", X_extra.shape)

assert X_raw.shape[1] == 64, "原始像素特征维度应为 64"
assert X_extra.shape[1] == 160, "扩展特征维度应为 160"

test_scores = np.array([[1.0, 2.0, 3.0],
                        [1.0, 1.0, 1.0]])
test_probs = softmax(test_scores)
print("test_probs:")
print(test_probs)
print("row sums:", test_probs.sum(axis=1))

assert np.allclose(test_probs.sum(axis=1), 1.0), "softmax 每一行概率之和应为

X_small = np.array([[1.0, 2.0],
                    [0.5, -1.0],
                    [1.5, 0.0]])
W_small = np.zeros((2, 3))
b_small = np.zeros(3)
y_small = np.array([0, 1, 2])
scores_small = X_small @ W_small + b_small

svm_loss, svm_dW, svm_db = svm_loss_and_gradient(
    X_small, scores_small, y_small, W_small, reg=0.0, delta=1.0
)

print("SVM loss on zero scores:", svm_loss)
print("svm_dW shape:", svm_dW.shape)
print("svm_db shape:", svm_db.shape)

assert svm_dW.shape == W_small.shape, "SVM dW shape 不正确"
assert svm_db.shape == b_small.shape, "SVM db shape 不正确"

```

```

In [ ]: def run_experiment(classifier_cls, use_extra_features, reg=1e-4, lr=0.1,
    X = build_features(images, use_extra_features=use_extra_features)

    X_train, X_test, y_train, y_test = train_test_split(

```

```

        X, labels, test_size=0.25, random_state=42, stratify=labels
    )

    X_train, X_test = standardize_train_test(X_train, X_test)

    clf = classifier_cls(
        lr=lr,
        reg=reg,
        epochs=epochs,
        batch_size=batch_size,
        random_state=42
    )
    clf.fit(X_train, y_train)

    train_acc = clf.accuracy(X_train, y_train)
    test_acc = clf.accuracy(X_test, y_test)
    weight_norm = np.linalg.norm(clf.W)

    return clf, train_acc, test_acc, weight_norm

# 实验 1: 比较原始像素特征和扩展统计特征
configs = [
    ("Softmax", SoftmaxClassifier, 0.5),
    ("SVM", SVMClassifier, 0.05),
]

feature_results = []

for model_name, classifier_cls, lr in configs:
    for use_extra in [False, True]:
        clf, train_acc, test_acc, weight_norm = run_experiment(
            classifier_cls=classifier_cls,
            use_extra_features=use_extra,
            reg=1e-4,
            lr=lr,
            epochs=120,
            batch_size=512
        )
        feature_name = "raw + structure" if use_extra else "raw pixels"
        feature_results.append((model_name, feature_name, train_acc, test

print("Feature comparison:")
print(f"{'model':>10} | {'features':>16} | {'train acc':>10} | {'test acc'
print("-" * 78)
for model_name, feature_name, train_acc, test_acc, weight_norm, _ in feat
    print(f"{'model_name':>10} | {'feature_name':>16} | {'train_acc:10.4f} | {'

# 可视化两个分类器的 loss 曲线
plt.figure(figsize=(7, 4))
for model_name, feature_name, train_acc, test_acc, weight_norm, clf in fe
    if feature_name == "raw + structure":
        plt.plot(clf.loss_history, label=model_name)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss Curves on Raw + Structure Features")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

```
In [ ]: # 实验 2: 比较不同 L2 正则强度下 Softmax 与 SVM 的表现
regs = [0.0, 1e-4, 1e-2, 1.0]
reg_results = []

for reg in regs:
    softmax_clf, train_acc, test_acc, weight_norm = run_experiment(
        classifier_cls=SoftmaxClassifier,
        use_extra_features=True,
        reg=reg,
        lr=0.5,
        epochs=120,
        batch_size=256
    )
    reg_results.append(("Softmax", reg, train_acc, test_acc, weight_norm))

    svm_clf, train_acc, test_acc, weight_norm = run_experiment(
        classifier_cls=SVMClassifier,
        use_extra_features=True,
        reg=reg,
        lr=0.05,
        epochs=120,
        batch_size=256
    )
    reg_results.append(("SVM", reg, train_acc, test_acc, weight_norm))

print("Regularization comparison:")
print(f"{'model':>10} | {'reg':>10} | {'train acc':>10} | {'test acc':>10}")
print("-" * 68)
for model_name, reg, train_acc, test_acc, weight_norm in reg_results:
    print(f"{'model_name':>10} | {reg:10.4g} | {train_acc:10.4f} | {test_ac
```